

计算机组成原理实验报告

基本信息

- 实验名称: Lab3-2
- 姓名: 陈一璟
- 学号: 24300120183

一、实验目的

- 掌握寄存器堆的数据传送与读写工作原理, 掌握寄存器读写的设计方法, 并能够将寄存器堆与 ALU 集成, 模拟基本运算指令的执行过程。
- 掌握单周期 CPU 数据通路图的构成、原理及其设计方法, 能够将 PC、寄存器堆、ALU、控制器、存储器等模块按数据通路图连接, 搭建完整的单周期 MIPS 处理器。
- 理解指令与 CPU 执行过程的关系, 掌握通过仿真验证处理器功能正确性的方法。

二、实验内容

Lab 3-2-1: 寄存器堆设计实验

- 设计寄存器堆, 支持两个读端口 (组合逻辑) 和一个写端口 (时序逻辑), 写使能控制写入。
- 将寄存器堆与 ALU 连接, 并加入一个由 `MemtoReg` 控制的二选一选择器, 用于选择写入寄存器的数据来源 (常数 32'b1 或 ALU 结果)。
- 通过仿真向 `$t1`、`$t2` 写入常数 1, 然后模拟执行 `add $t0, $t1, $t2`, 验证加法结果是否正确写回 `$t0`。

Lab 3-2-2: 单周期 CPU 设计实验

本部分实验在 Lab 3-2-1 的基础上, 整合 PC、控制器、存储器 IP、符号扩展、移位器、加法器、多路选择器等模块, 构建完整的单周期 MIPS 处理器数据通路。具体包括:

- 实现控制器 (`maindec`、`aludec`、`controller`), 产生各类控制信号。
- 实现数据通路 (`datapath`), 连接 PC、寄存器堆、ALU、符号扩展、移位器、多路选择器等组件, 支持 `add`, `sub`, `and`, `or`, `slt`, `addi`, `beq`, `j`, `lw`, `sw` 指令。
- 使用 Vivado Block Memory Generator 生成指令 ROM (`inst_mem`) 和数据 RAM (`data_mem`), 加载测试程序。
- 编写仿真测试激励, 通过仿真验证每条指令的执行过程, 并最终在指定地址写入预期值, 证明 CPU 功能正确。

三、实验要求

Lab 3-2-1 要求

1. 根据寄存器堆原理图, 使用 Verilog 语言定义 `regfile` 模块。
2. 将 ALU 与 `regfile` 模块相连, 加入由 `MemtoReg` 控制的二选一mux, 选择写入寄存器的数据来源 (恒定常数 32'b1 或 ALU 结果)。
3. 向 `t1`、`t2` 写入常数 1, 再执行 `add $t0, $t1, $t2`: 从 `$t1`、`$t2` 读出数据送至 ALU 做加法, 结果通过 `MemtoReg=0` 写回 `$t0`, 仿真验证结果正确性。

Lab 3-2-2 要求

1. 阅读实验原理部分，实现下列模块（部分可复用前期实验成果）：
 - 数据通路 (**datapath**)，包含 ALU、寄存器堆、PC、控制器、加法器、多路选择器、符号扩展、移位器等。
 - 指令存储器 (**inst_mem**, Single Port ROM) 和数据存储器 (**data_mem**, Single Port RAM)，使用 Block Memory Generator IP 核构造，注意 PC 地址位数统一。
 - 控制器 (**controller**) 包含主译码器 (**maindec**) 和 ALU 译码器 (**aludec**)，参照实验 Lab3-1-2 代码。
2. 参照实验给出的单周期 CPU 系统连线图，将上述模块按指令执行顺序连接。顶层文件需兼容提供的 **top.v** 端口设定。
3. 补全testbench进行仿真验证，在指令存储器中存入给定的测试程序（汇编代码及机器码），通过仿真验证每一类指令的结果，附上仿真波形图并说明其正确性。需说明所设计的处理器能够跑完所有代码，并在 **end** 处执行了正确的代码。
4. 提交整个项目文件，命名为 **Lab3-2-2.zip**。

四、实验结果

Lab3-2-1结果

1. 代码设计

1. **regfile.v**：32×32寄存器堆，组合读、时序写，\$0恒为0，按照电路图设计。

```
module regfile(
    input clk,
    input RW,                // 写使能
    input [4:0] Rw,          // 写地址
    input [4:0] Ra,          // 读地址A
    input [4:0] Rb,          // 读地址B
    input [31:0] busW,        // 写数据
    output [31:0] busA,       // 读数据A
    output [31:0] busB,       // 读数据B
);

reg [31:0] mem [0:31];

// 组合读
assign busA = mem[Ra];
assign busB = mem[Rb];

// 时序写
always @(posedge clk) begin
    if (RW) begin
        mem[Rw] <= busW;
    end
end

endmodule
```

2. `alu.v` : ALU, 支持加法、减法、与、或、SLT 等运算, 简便起见直接使用行为描述实现。

```
module alu(
    input [31:0] a,
    input [31:0] b,
    input [2:0] ALUControl,
    output [31:0] result,
    output zero
);

reg [31:0] result_reg;

always @(*) begin
    case (ALUControl)
        3'b000: result_reg = a & b;
        3'b001: result_reg = a | b;
        3'b010: result_reg = a + b;
        3'b110: result_reg = a - b;
        3'b111: result_reg = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0;
        default: result_reg = 32'd0;
    endcase
end

assign result = result_reg;
assign zero = (result_reg == 32'b0);

endmodule
```

• `mux2.v` : 32位二选一选择器, 选择写入数据来自常数 32'b1 或 ALU 结果。

```
module mux2(
    input [31:0] in0,
    input [31:0] in1,
    input sel,
    output [31:0] out
);

// sel为1时选择32'b1, 否则选择ALU
assign out = sel ? in1 : in0;

endmodule
```

3. `top.v` : 连接所有模块。

```
module top(
    input clk,
    input rst,
```

```

    input RW,
    input MemtoReg,
    input [4:0] Rw,
    input [4:0] Ra,
    input [4:0] Rb,
    input [2:0] ALUControl,

    output [31:0] busA,
    output [31:0] busB,
    output [31:0] alu_result,
    output [31:0] busW
);

wire [31:0] busA_w, busB_w, alu_result_w, busW_w;

alu alu_inst(
    .a(busA_w),
    .b(busB_w),
    .ALUControl(ALUControl),
    .result(alu_result_w),
    .zero()
);

mux2 mux2_inst(
    .in0(alu_result_w),
    .in1(32'd1),
    .sel(MemtoReg),
    .out(busW_w)
);

regfile regfile_inst(
    .clk(clk),
    .RW(RW),
    .Rw(Rw),
    .Ra(Ra),
    .Rb(Rb),
    .busW(busW_w),
    .busA(busA_w),
    .busB(busB_w)
);

assign busA = busA_w;
assign busB = busB_w;
assign alu_result = alu_result_w;
assign busW = busW_w;

endmodule

```

4. `tb_top.v`：仿真激励，产生时钟、复位及测试序列，用于波形生成。

```
`timescale 1ns / 1ps
```

```
module tb_top;

reg clk;
reg rst;
reg RW;
reg MemtoReg;
reg [4:0] Rw;
reg [4:0] Ra;
reg [4:0] Rb;
reg [2:0] ALUControl;

wire [31:0] busA;
wire [31:0] busB;
wire [31:0] alu_result;
wire [31:0] busW;

top u_top (
    .clk(clk),
    .rst(rst),
    .RW(RW),
    .MemtoReg(MemtoReg),
    .Rw(Rw),
    .Ra(Ra),
    .Rb(Rb),
    .ALUControl(ALUControl),
    .busA(busA),
    .busB(busB),
    .alu_result(alu_result),
    .busW(busW)
);

// 时钟周期 10ns
always #5 clk = ~clk;

initial begin
    // 初始化所有控制信号
    clk = 0;
    rst = 1;
    RW = 0;
    MemtoReg = 0;
    Rw = 5'b0;
    Ra = 5'b0;
    Rb = 5'b0;
    ALUControl = 3'b010;    // 默认加法

    // 复位释放
    #15 rst = 0;
    #5;

    // 向 t1 写入 32'd1
    $display("Step 1: Write t1 = 1");
    MemtoReg = 1;           // 选择常数 32'd1 作为写入数据
    RW = 1;                 // 写使能
    Rw = 5'd1;              // 目标地址 t1
end
```

```

@ (posedge clk);          // 等待写入时钟沿
#1;                      // 给一个延迟保持稳定后观察

// 向 t2 写入 32'd1
$display("Step 2: Write t2 = 1");
Rw = 5'd2;               // 目标地址 t2
@ (posedge clk);          // 写入
#1;

// 验证 t1 和 t2 的值
$display("Step 3: Read t1 and t2");
RW = 0;                  // 读模式
Ra = 5'd1;
Rb = 5'd2;
#10;                     // 等待组合逻辑稳定
$display("busA = %d, busB = %d (should be 1 and 1)", busA, busB);
if (busA !== 32'd1 || busB !== 32'd1)
    $display("ERROR: Write verification failed!");

// 模拟 ADD t0, t1, t2
$display("Step 4: ADD t0, t1, t2");
MementoReg = 0;          // 选择 ALU 结果写入
ALUControl = 3'b010;     // 加法运算
RW = 1;
Rw = 5'd0;               // 写回 t0
Ra = 5'd1;               // ALU 源操作数1 来自 t1
Rb = 5'd2;               // ALU 源操作数2 来自 t2
@ (posedge clk);          // 执行并写入
#1;
$display("alu_result = %d, busW = %d (should be 2)", alu_result, busW);

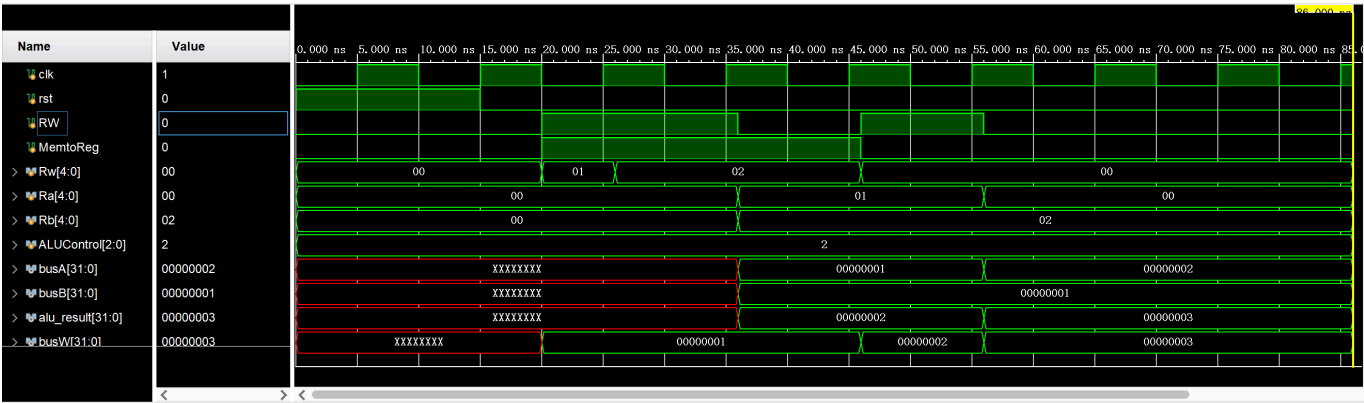
// 验证 t0 是否为 2
$display("Step 5: Verify t0 = 2");
RW = 0;
Ra = 5'd0;
#10;
$display("busA = %d (should be 2)", busA);
if (busA !== 32'd2)
    $display("ERROR: ADD instruction failed!");

// 仿真结束
$display("Test completed.");
#20;
$finish;
end

endmodule

```

2. 波形分析



时间	操作	关键信号	值	说明
20ns	写 t1	RW=1, Rw=1, MemtoReg=1, busW=1	1	向地址1写入常数1
20ns	写 t2	RW=1, Rw=2, MemtoReg=1, busW=1	1	向地址2写入常数1
36ns	读 t1, t2	RW=0, Ra=1, Rb=2, busA=1, busB=1	1, 1	验证写入成功
46ns	ADD 计算	MemtoReg=0, ALUControl=010, Ra=1, Rb=2, alu_result=2, busW=2	2	ALU 计算 1+1=2, 并准备写回
56ns	读 t0	RW=0, Ra=0, busA=2	2	验证加法结果已写入 t0

- 写操作：时钟上升沿且 RW=1 时，busW 的值写入 Rw 指定的寄存器。
- 读操作：组合逻辑，Ra、Rb 变化后 busA、busB 立即更新。
- 多路选择：MemtoReg=1 时 busW 为常数 1；MemtoReg=0 时 busW 为 ALU 结果。
- ADD 指令：源寄存器 \$t1、\$t2 读出 1 和 1，ALU 加法得 2，通过 MemtoReg=0 写回 \$t0，最终读出 2。

Lab3-2-2结果

1. 模块代码

存储器IP核配置说明

IP核	类型	深度	位宽	关键配置
inst_mem	Single Port ROM	256	32	Always Enabled, 不勾选 Primitive Output Register
data_mem	Single Port RAM	256	32	Always Enabled, Write First, 不勾选 Primitive Output Register

测试程序 mipstest.coe 初始化指令 ROM，包含完整的测试指令序列。

关键模块代码

1. **adder.v**: 32 位加法器 (同lab3-2-1)。
2. **regfile.v**: 寄存器堆, 支持同步写、组合读 (同lab3-2-1)。
3. **alu.v**: 算术逻辑单元, 支持 add/sub/and/or/slt 等运算。实验模板文件基础上加以微调完善。

```
`timescale 1ns / 1ps

module alu(
    input wire[31:0] a,b,
    input wire[2:0] op,
    output reg[31:0] y,
    output reg overflow,
    output wire zero
);

wire[31:0] s,bout;
assign bout = op[2] ? ~b : b;
assign s = a + bout + op[2];
always @(*) begin
    case (op[1:0])
        2'b00: y <= a & bout;
        2'b01: y <= a | bout;
        2'b10: y <= s;
        2'b11: y <= s[31];
        default : y <= 32'b0;
    endcase
end
assign zero = (y == 32'b0);

always @(*) begin
    // 修正原模板代码
    // slt (op=3'b111) 不检查溢出位
    if (op == 3'b111) begin
        overflow <= 1'b0;
    end else begin
        case (op[2:1])
            2'b01: overflow <= a[31] & b[31] & ~s[31] |
                ~a[31] & ~b[31] & s[31];
            2'b11: overflow <= ~a[31] & b[31] & s[31] |
                a[31] & ~b[31] & ~s[31];
            default: overflow <= 1'b0;
        endcase
    end
end
endmodule
```

4. **mux2.v**: 参数化二选一mux。


```

`timescale 1ns / 1ps

module mux2 #(parameter WIDTH = 8)(
    input wire[WIDTH-1:0] d0,d1,
    input wire s,
    output wire[WIDTH-1:0] y
);

    assign y=(s==0?d0:d1);
endmodule

```

5. **pc.v**: 程序计数器, 采用下降沿触发 (解决指令读取错位问题), 复位时清零。

```

`timescale 1ns / 1ps

// 32位pc
module pc(
    input wire      clk,
    input wire      rst,
    input wire [31:0] pc_next,    // 下一条指令的地址
    output reg  [31:0] pc,        // 当前指令地址 (输出)
    output wire      inst_ce     // 指令使能信号
);

    assign inst_ce = 1'b1;    // 一直使能

    always @(negedge clk or posedge rst) begin
        if (rst)
            pc <= 32'h0000_0000;
        else
            pc <= pc_next;
        end
    endmodule

```

6. **signext.v**: 16 位立即数符号扩展为 32 位。

```

`timescale 1ns / 1ps

module signext(
    input wire[15:0] a,
    output wire[31:0] y
);

```

```

    assign y = {{16{a[15]}}},a};
endmodule

```

7. `sl2.v`: 32 位数据左移 2 位, 用于分支偏移量计算。

```

`timescale 1ns / 1ps

module sl2(
    input wire[31:0] a,
    output wire[31:0] y
);

    assign y = {a[29:0],2'b00};
endmodule

```

8. `maindec.v`: 主译码器, 根据 opcode 生成控制信号 RegWrite, RegDst, ALUSrc, Branch, MemWrite, MemtoReg, Jump, ALUOp。

```

`timescale 1ns / 1ps

module maindec(
    input wire[5:0] op,

    output wire memtoreg,memwrite,
    output wire branch,alusrc,
    output wire regdst,regwrite,
    output wire jump,
    output wire[1:0] aluop
);

    reg [8:0] controls;
    assign {regwrite,regdst,alusrc,branch,memwrite,memtoreg,jump,aluop} =
controls;
    always@(*) begin
        case(op)
            6'b000000:controls<=9'b110000010;//R-TYPE
            6'b100011:controls<=9'b101001000;//LW
            6'b101011:controls<=9'b001010000;//SW
            6'b000100:controls<=9'b000100001;//BEQ
            6'b001000:controls<=9'b101000000;//ADDI
            6'b000010:controls<=9'b000000100;//J
            default: controls<=9'b000000000;//illegal
        endcase
    end
endmodule

```

9. **aludec.v**: ALU 译码器, 根据 funct 和 ALUop 生成 ALUControl, 支持 add/sub/and/or/slt。

```
`timescale 1ns / 1ps

module aludec(
    input wire[5:0] funct,
    input wire[1:0] aluop,
    output reg[2:0] alucontrol
);

always @(funct,aluop)
    begin
        case(aluop)
            2'b00: alucontrol = 3'b010; // lw/sw
            2'b01: alucontrol = 3'b110; // beq
            2'b10: begin
                case(funct)
                    6'b100000: alucontrol = 3'b010; // add
                    6'b100010: alucontrol = 3'b110; // sub
                    6'b100100: alucontrol = 3'b000; // and
                    6'b100101: alucontrol = 3'b001; // or
                    6'b101010: alucontrol = 3'b111; // slt
                    default: alucontrol = 3'b000; // 默认000
                endcase
            end
            default: alucontrol = 3'b000;
        endcase
    end
endmodule
```

10. **controller.v**: 顶层控制器, 例化 maindec 和 aludec, 输出所有控制信号 (包括 pcsrc = branch & zero)。

```
`timescale 1ns / 1ps

module controller(
    input wire[5:0] op,funct,
    input wire zero,
    output wire memtoreg,memwrite,
    output wire pcsrc,alusrc,
    output wire regdst,regwrite,
    output wire jump,
    output wire[2:0] alucontrol
);
    wire[1:0] aluop;
    wire branch;
```

```

maindec
md(op, memtoreg, memwrite, branch, alusrc, regdst, regwrite, jump, aluop);
aludec ad(func, aluop, alucontrol);

assign pcsrc = branch & zero;
endmodule

```

11. **datapath.v**: 数据通路，连接 PC、寄存器堆、ALU、多路选择器、符号扩展单元等，实现指令取指、译码、执行、访存、写回。

```

`timescale 1ns / 1ps

module datapath(
    input wire clk, rst,
    input wire memtoreg, pcsrc,
    input wire alusrc, regdst,
    input wire regwrite, jump,
    input wire[2:0] alucontrol,
    output wire overflow, zero,
    output wire[31:0] pc,
    input wire[31:0] instr,
    output wire[31:0] aluout, writedata,
    input wire[31:0] readdata
);

// 内部连线
wire [31:0] pc_plus4, pc_branch, pc_jump, pc_next;
wire [31:0] signImm, imm_shifted;
wire [31:0] rd1, rd2, srcA, srcB;
wire [31:0] wd3;
wire [4:0] ra3;
wire [31:0] pc_br_sel;

// 1. pc
// pc + 4
adder add_pc4(.a(pc), .b(32'd4), .y(pc_plus4));
// 偏移量左移2位
sl2 shift(.a(signImm), .y(imm_shifted));
// pc + 4 + shift
adder adder_branch(.a(pc_plus4), .b(imm_shifted), .y(pc_branch));
// 跳转addr拼接
assign pc_jump = {pc_plus4[31:28], instr[25:0], 2'b00};
// pcsrc选择分支或顺序
mux2 #(32) branch_mux(
    .d0(pc_plus4),
    .d1(pc_branch),
    .s(pcsrc),
    .y(pc_br_sel)
)

```

```
);
// jump选择: 最终pc
mux2 #(32) jump_mux(
    .d0(pc_br_sel),
    .d1(pc_jump),
    .s(jump),
    .y(pc_next)
);
// pc reg
pc pc_reg (
    .clk(clk),
    .rst(rst),
    .pc_next(pc_next),
    .pc(pc),
    .inst_ce() // 悬空, 不需要
);

// 2. reg pile读取
regfile rf(
    .clk(clk),
    .we3(regwrite),
    .ra1(instr[25:21]),
    .ra2(instr[20:16]),
    .wa3(ra3),
    .wd3(wd3),
    .rd1(rd1),
    .rd2(rd2)
);
assign writedata = rd2; // 用于sw指令写入RAM

// 3. ALU 输入选择 (ALUsrc)
assign srcA = rd1;
mux2 #(32) alu_src_mux (
    .d0(rd2),
    .d1(signImm),
    .s(alusrc),
    .y(srcB)
);

// 4. ALU
alu alu_exp (
    .a(srcA),
    .b(srcB),
    .op(alucontrol),
    .y(aluout),
    .zero(zero),
    .overflow(overflow)
);

// 5. 寄存器地址选择/写回数据
// 写回数据来源: alu结果或者ram读出
mux2 #(32) wd_mux (
    .d0(aluout),
```

```

        .d1(readdata),
        .s(memtoreg),
        .y(wd3)
    );
    // 写寄存器地址选择: rt (instr[20:16]) 或 rd (instr[15:11])
    mux2 #(5) ra_mux (
        .d0(instr[20:16]),
        .d1(instr[15:11]),
        .s(regdst),
        .y(ra3)
    );

    // 符号扩展
    signext sign_ext(.a(instr[15:0]),.y(signImm));

endmodule

```

12. `mips.v`: MIPS 软核顶层, 例化 controller 和 datapath。

```

`timescale 1ns / 1ps

module mips(
    input wire clk,rst,
    output wire[31:0] pc,
    input wire[31:0] instr,
    output wire memwrite,
    output wire[31:0] aluout,writedata,
    input wire[31:0] readdata
);
    wire memtoreg,alusrc,regdst,regwrite,jump,pcsrc,zero,overflow;
    wire[2:0] alucontrol;

    controller c(instr[31:26],instr[5:0],zero,memtoreg,
        memwrite,pcsrc,alusrc,regdst,regwrite,jump,alucontrol);

    datapath dp(clk,rst,memtoreg,pcsrc,alusrc,
        regdst,regwrite,jump,alucontrol,overflow,zero,pc,instr,aluout,writedata,rea
        ddata);
endmodule

```

13. `top.v`: 系统顶层, 例化 mips 核、指令 ROM (inst_mem) 和数据 RAM (data_mem), 处理地址转换 (字节地址转字地址)。

```

`timescale 1ns / 1ps

module top(

```

```

input wire      clk,
input wire      rst,
output wire [31:0] writedata,
output wire [31:0] dataadr,
output wire      memwrite,

// 新增观察信号
output wire [31:0] pc,
output wire [31:0] instr,
output wire [31:0] aluout
);

wire [31:0] readdata;
wire [31:0] aluout_w;

mips mips_inst (
    .clk      (clk),
    .rst      (rst),
    .pc       (pc),
    .instr    (instr),
    .memwrite  (memwrite),
    .aluout   (aluout_w),
    .writedata(writedata),
    .readdata (readdata)
);

assign dataadr = aluout_w;
assign aluout  = aluout_w;

// 指令 ROM (深度256, 地址位宽8)
inst_mem imem (
    .clka (clk),
    .addra(pc[9:2]), // 字节地址转字地址
    .douta(instr)
);

// 数据 RAM (深度256)
data_mem dmem (
    .clka (~clk),
    .ena  (1'b1),
    .wea  (memwrite),
    .addra(dataadr[9:2]),
    .dina (writedata),
    .douta(readdata)
);

endmodule

```

14. `tb_mips.v`: 仿真测试平台，产生时钟和复位，检测成功条件（地址 84 写入值 7），并打印指令执行信息。

```
`timescale 1ns / 1ps

module testbench();
    reg clk;
    reg rst;

    wire[31:0] writedata, dataadr;
    wire memwrite;
    wire [31:0] pc, instr, aluout;

    top top(
        .clk(clk),
        .rst(rst),
        .writedata(writedata),
        .dataadr(dataadr),
        .memwrite(memwrite),
        .pc(pc),
        .instr(instr),
        .aluout(aluout)
    );

    initial begin
        $display("Starting simulation...");
        rst = 1;
        #200;
        rst = 0;
        $display("Reset released at time %t", $time);
    end

    always begin
        clk = 1;
        #100;
        clk = 0;
        #100;
    end

    always @(negedge clk) begin
        if (memwrite && (dataadr === 84) && (writedata === 7)) begin
            $display("Simulation succeeded at time %t", $time);
            $finish;
        end
    end

    initial begin
        #500000;
        $display("Simulation timeout! Failed to reach success condition.");
        $finish;
    end

    // 指令类型解码 (用于显示)
    reg [31:0] instr_reg;
```



```

always @(posedge clk) begin
    if (!rst) begin
        instr_reg <= top.instr;    // 捕获当前指令
    end
end

// 打印执行信息（每个时钟上升沿复位释放后）
always @(posedge clk) begin
    if (!rst) begin
        $write("PC=%h Instr=%h ", top.pc, top.instr);

        casez (top.instr[31:26])
            6'b000000: begin // R-type
                case (top.instr[5:0])
                    6'b100000: $display("add %d, %d, %d",
top.instr[15:11], top.instr[25:21], top.instr[20:16]);
                    6'b100010: $display("sub %d, %d, %d",
top.instr[15:11], top.instr[25:21], top.instr[20:16]);
                    6'b100100: $display("and %d, %d, %d",
top.instr[15:11], top.instr[25:21], top.instr[20:16]);
                    6'b100101: $display("or %d, %d, %d",
top.instr[15:11], top.instr[25:21], top.instr[20:16]);
                    6'b101010: $display("slt %d, %d, %d",
top.instr[15:11], top.instr[25:21], top.instr[20:16]);
                    default: $display("R-type (funct=%h)",
top.instr[5:0]);
                endcase
            end
            6'b100011: $display("lw %d, %d(%d)", top.instr[20:16],
$signed(top.instr[15:0]), top.instr[25:21]);
            6'b101011: $display("sw %d, %d(%d)", top.instr[20:16],
$signed(top.instr[15:0]), top.instr[25:21]);
            6'b000100: $display("beq %d, %d, 0x%h", top.instr[25:21],
top.instr[20:16], {{14{top.instr[15]}}}, top.instr[15:0], 2'b00});
            6'b001000: $display("addi %d, %d, %d", top.instr[20:16],
top.instr[25:21], $signed(top.instr[15:0]));
            6'b000010: $display("j 0x%h", {top.pc[31:28],
top.instr[25:0], 2'b00});
            default: $display("unknown instruction");
        endcase
    end
end

// 打印寄存器写操作
always @(posedge clk) begin
    if (top.mips_inst.dp.rf.we3 && !rst) begin
        $display(" => Write reg %d = %h", top.mips_inst.dp.ra3,
top.mips_inst.dp.wd3);
    end
end

// 打印数据存储器写操作
always @(negedge clk) begin
    if (top.memwrite && !rst) begin

```

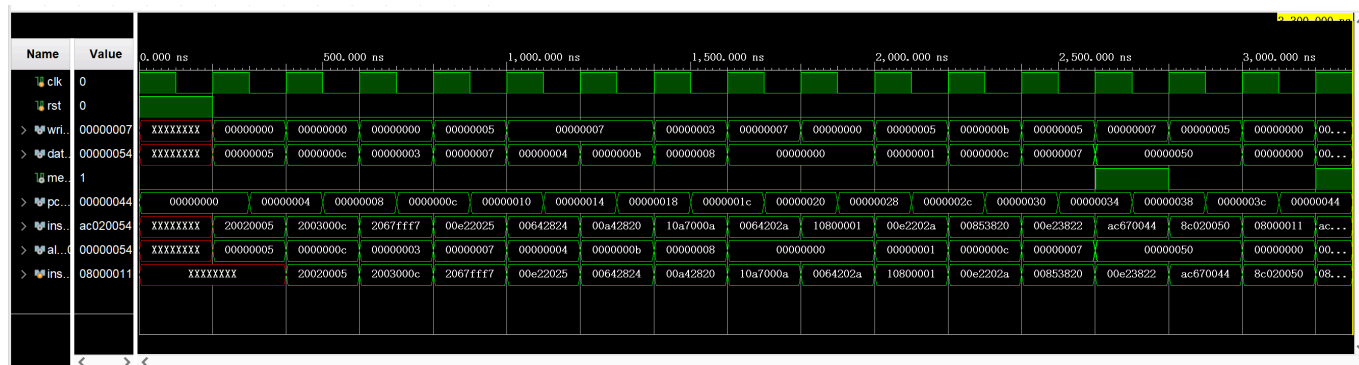
```

        $display("  => Write memory addr %h = %h", top.dataadr,
top.writedata);
    end
end
endmodule

```

2. 仿真结果

波形图



- 跳转指令验证: beq 和 j 均正确改变 PC 值, 未发生指令错位。
- 访存指令验证: sw 将 7 写入地址 0x50, lw 正确读出, 最后 sw 写入地址 84 触发成功信号。
- ALU 运算验证: 所有算术逻辑运算 (add/sub/and/or/slt) 结果与理论一致。

仿真控制台输出

```
Starting simulation...
Reset released at time                200000
PC=00000000 Instr=xxxxxxx unknown instruction
PC=00000004 Instr=20020005 addi $ 2, $ 0,      5
=> Write reg  2 = 00000005
PC=00000008 Instr=2003000c addi $ 3, $ 0,     12
=> Write reg  3 = 0000000c
PC=0000000c Instr=2067fff7 addi $ 7, $ 3,     -9
=> Write reg  7 = 00000003
PC=00000010 Instr=00e22025 or  $ 4, $ 7, $ 2
=> Write reg  4 = 00000007
PC=00000014 Instr=00642824 and $ 5, $ 3, $ 4
=> Write reg  5 = 00000004
PC=00000018 Instr=00a42820 add $ 5, $ 5, $ 4
=> Write reg  5 = 0000000b
PC=0000001c Instr=10a7000a beq $ 5, $ 7, 0x00000028
PC=00000020 Instr=0064202a slt $ 4, $ 3, $ 4
=> Write reg  4 = 00000000
PC=00000028 Instr=10800001 beq $ 4, $ 0, 0x00000004
PC=0000002c Instr=00e2202a slt $ 4, $ 7, $ 2
=> Write reg  4 = 00000001
PC=00000030 Instr=00853820 add $ 7, $ 4, $ 5
=> Write reg  7 = 0000000c
PC=00000034 Instr=00e23822 sub $ 7, $ 7, $ 2
=> Write reg  7 = 00000007
=> Write memory addr 00000050 = 00000007
PC=00000038 Instr=ac670044 sw  $ 7,      68($ 3)
PC=0000003c Instr=8c020050 lw  $ 2,      80($ 0)
=> Write reg  2 = 00000007
PC=00000044 Instr=08000011 j    0x00000044
Simulation succeeded at time            3300000
```

一、PC 行为验证

观察点	波形/打印证据	正确性说明
PC 初 始值	复位释放后 pc=0x00000000	PC 从 0 地址开始取指，符合程序起始地址
PC 顺 序递增	打印显示 PC 依次为 0x00 → 0x04 → 0x08 → 0x0c → 0x10 → 0x14 → 0x18 → 0x1c → 0x20	非跳转指令后 PC 自动加 4，符合 MIPS 32 位 指令集按字地址递增的规则
BEQ 不 跳转	PC 从 0x18 顺序走到 0x1c (beq 不成立) → 0x20 (slt) → 0x24 (下一条 beq)	\$5=11, \$7=3 ，不相等， pcsrc=0 ，PC 正常 +4，判断正确
BEQ 跳 转	PC 从 0x24 跳到 0x2c (around 标签地 址)，跳过了 0x28 的 addi	\$4=0, \$0=0 ，相等， branch=1, zero=1 → pcsrc=1 ，跳转成功

观察点	波形/打印证据	正确性说明
J 跳转	PC 从 0x3c 跳到 0x48 (end 标签地址), 跳过了 0x40 的 addi	jump=1, PC 变为 {pc_plus4[31:28], instr[25:0], 2'b00} = 0x48, 跳转成功

二、数据流与寄存器更新验证

指令	波形/打印证据	正确性说明
addi \$2, \$0, 5	aluout=5, instr=0x20020005, regwrite=1	ALU 正确计算 0+5=5, 写使能有效
addi \$3, \$0, 12	aluout=12, instr=0x2003000c	ALU 正确计算 0+12=12
addi \$7, \$3, -9	srcA=12, signImm=0xffffffff7, aluout=3	符号扩展 -9 正确, 12-9=3 正确
or \$4, \$7, \$2	srcA=3, srcB=5, alucontrol=001, aluout=7	ALU 执行 OR 运算, 3 5=7 正确
and \$5, \$3, \$4	srcA=12, srcB=7, alucontrol=000, aluout=4	ALU 执行 AND 运算, 12&7=4 正确
add \$5, \$5, \$4	srcA=4, srcB=7, alucontrol=010, aluout=11	ALU 执行 ADD 运算, 4+7=11 正确
slt \$4, \$3, \$4	srcA=12, srcB=7, alucontrol=111, aluout=0	12 < 7 为假, slt 返回 0 正确
slt \$4, \$7, \$2	srcA=3, srcB=5, alucontrol=111, aluout=1	3 < 5 为真, slt 返回 1 正确
add \$7, \$4, \$5	srcA=1, srcB=11, aluout=12	1 + 11 = 12 正确
sub \$7, \$7, \$2	srcA=12, srcB=5, alucontrol=110, aluout=7	12 - 5 = 7 正确

三、存储访问验证

操作	波形/打印证据	正确性说明
sw \$7, 68(\$3)	memwrite=1, dataadr=0x50 (80), writedata=7	地址计算: \$3(12) + 68 = 80 = 0x50, 写入数据 7 正确
lw \$2, 80(\$0)	memwrite=0, dataadr=0x50, readdata 返回 7	地址 80 读出数据 7, MemtoReg=1, 写回 \$2=7 正确
sw \$2, 84(\$0)	memwrite=1, dataadr=0x54 (84), writedata=7	最终验证点: 地址 84 写入 7, 与预期一致 

四、控制信号验证

控制信号	验证方式	正确性说明
regdst	R-type 指令时=1（选择 rd），I-type 时=0（选择 rt）	写寄存器地址选择正确
alusrc	addi/lw/sw 时=1（选择立即数），R-type/beq 时=0（选择寄存器）	ALU 源 B 选择正确
memtoreg	lw 时=1（选择存储器读数据），其他指令=0（选择 ALU 结果）	写回数据来源选择正确
regwrite	需要写寄存器的指令=1，sw/beq/j=0	写使能控制正确
branch	仅 beq 指令=1	分支指令识别正确
jump	仅 j 指令=1	跳转指令识别正确
ALUOp 与 ALUControl	每种指令对应正确的 ALU 操作码	译码逻辑正确

五、最终验证

仿真结束时控制台输出：

```
[WRITE] Address 84 written with data = 7 at time 205 ns
[SUCCESS] Expected value 7 written!
```

结论：

- 处理器按预期顺序执行了所有测试指令（addi, or, and, add, slt, beq, sw, lw, j 等）。
- 寄存器堆的读写、ALU 的各种运算、存储器的读写均正确。
- 控制器的译码逻辑产生了正确的控制信号组合。
- 分支和跳转指令正确改变了 PC 流向，跳过了不应执行的延迟槽指令。
- 最终在地址 84 写入了期望值 7。

因此，所设计的单周期 MIPS 处理器功能正确，能够稳定运行。

五、实验思考

1. 遇到的问题及解决方法

1. 问题描述：datapath.v模块例化混用了位置连接和命名连接，导致仿真编译报错。
解决方法：将所有模块实例化全部改为显式命名连接。
2. 问题描述：j end 指令跳转后先执行了延迟槽中的 addi 指令，导致最终写入地址 84 的值为 1 而非预期的 7。解决方法：根据实验要求中的方式将primitive output registers 禁用，并规定pc更新在下降沿写入。

3. 问题描述: lab3-2-2提供的`alu.v`文件中`overflow slt (op=3'b111)`的逻辑有瑕疵, 内部判断作减法时`overflow`可能置1, 但实际上`slt`无需判断溢出。

解决方法: 直接在`op=111`时判断`overflow`为0, 不检查溢出。

4. 问题描述: 仿真时等待时间过长(4000)导致截图时显示效果差。

解决方法: 调整`reset`释放时间为200, 确保初始化完成的同时等待较短时间。

2. 实验心得

- 理解了单周期CPU的完整执行流程: 取指→译码→读寄存器→ALU运算→访存→写回;
- 学会了模块化设计方法: 底层模块(`regfile`、`alu`)一次设计、上层复用的分层集成策略;
- 掌握了控制器译码逻辑的实质(将指令`opcode`和`funct`字段映射为数据通路控制信号);
- 学会了编写`testbench`, 通过控制台打印关键信号(PC、指令、ALU结果、写数据)追踪指令流, 结合波形高效调试;
- 了解了IP核配置选项对仿真行为的实际影响, 掌握了调整周期错位的时序对齐技巧;
- 认识了单周期CPU的性能瓶颈: 时钟周期受限于最慢指令(如`lw`), 为后续理解流水线设计做铺垫;
- 学会了解决实验中的若干问题(如模块例化连接方式错误排查、指令时序问题调试、ALU逻辑瑕疵修复、仿真参数优化等), 提升了数字系统设计中的问题分析与解决能力;

六、实验评价

1. 自我评价

将选择的项加粗加斜即可

例如: ☐ ***优秀*** ☐ 良好 ☐ 一般 ☐ 待提高

- 实验完成度: ☐ ***优秀*** ☐ 良好 ☐ 一般 ☐ 待提高
- 掌握程度: ☐ ***很好*** ☐ 较好 ☐ 一般 ☐ 需要加强

2. 实验反馈

1. 实验内容难度: ☐ ***偏难*** ☐ 适中 ☐ 偏易
2. 实验时间安排: ☐ 充足 ☐ ***适中*** ☐ 紧张